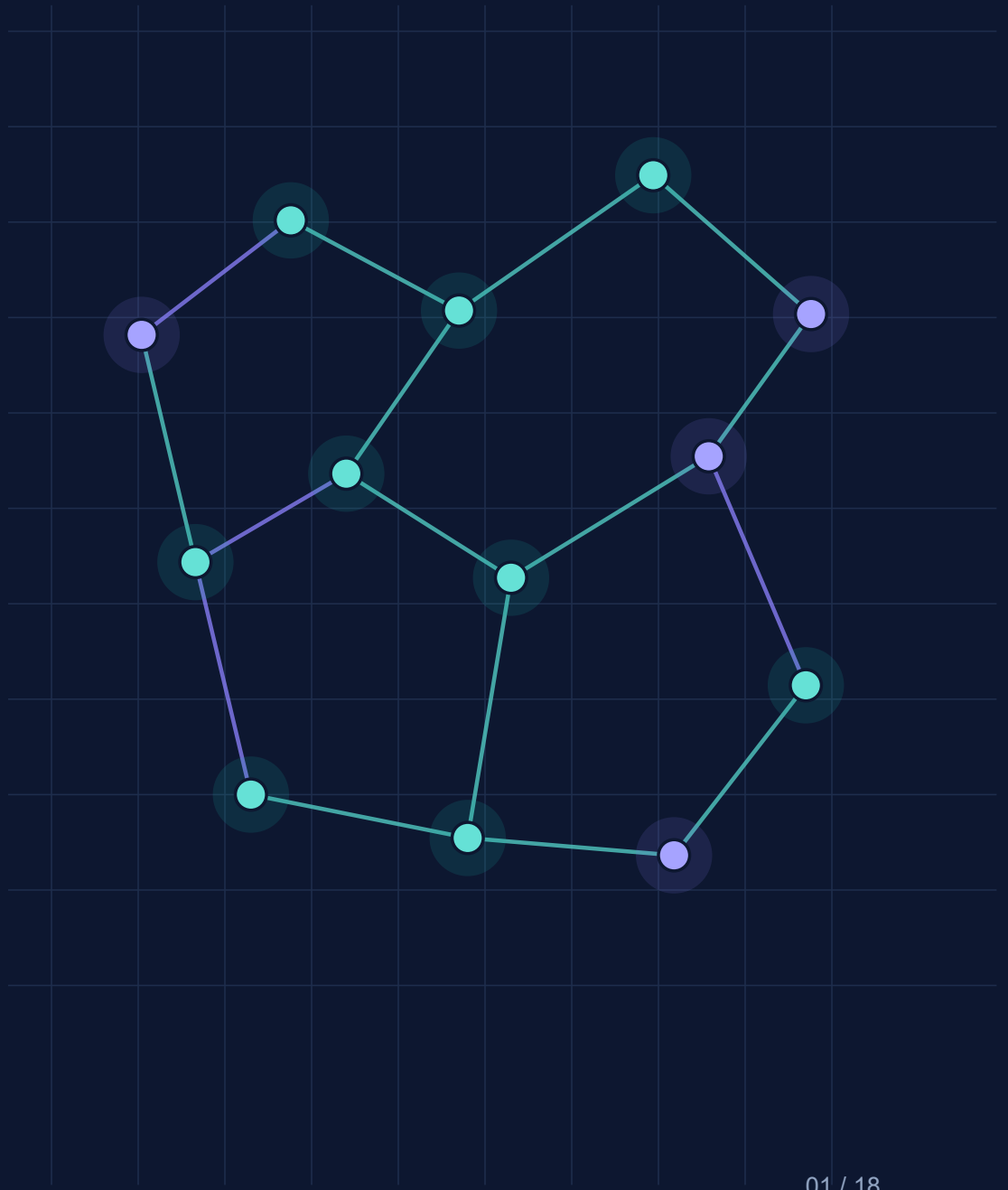
ASSIGNMENT 4 – Due Sunday, December 6th

From spatial points to useful graphs

A4 — Spatial Threshold Graphs

K-d trees · range queries · graph connectivity



THE CENTRAL QUESTION

How can we connect nearby points without testing every possible pair?

(These slides were adapted by ChatGPT to make my original ones more flashy)

Many datasets are collections of spatial points

Think of sensor sites, 3D scans, or positions of moving agents.

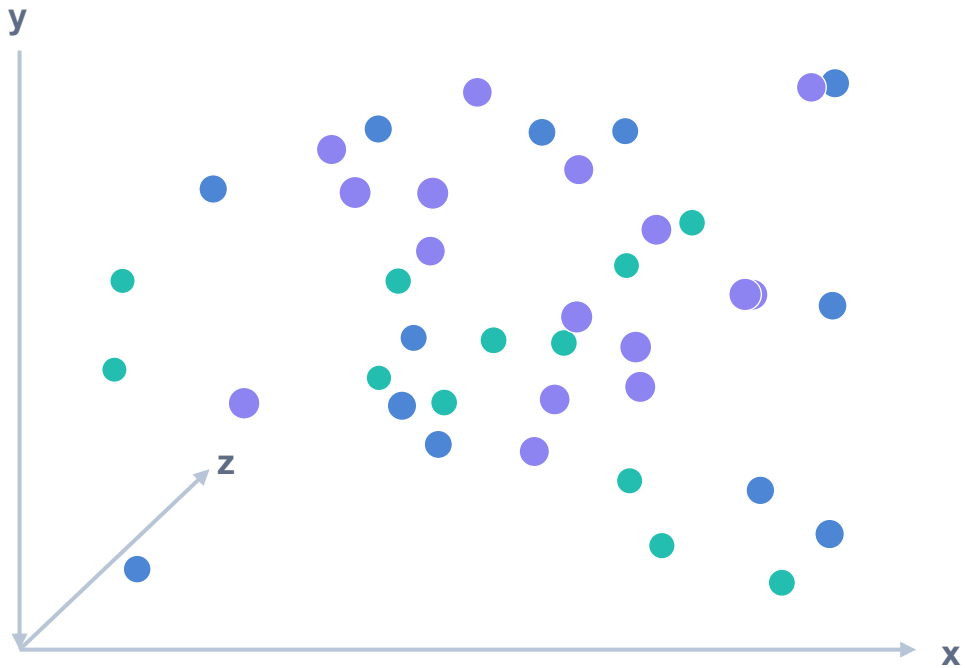
The raw data

Each input record is a point with x, y, and z coordinates.

The missing structure

A point cloud does not yet tell us which pairs should be neighbors.

Our goal: turn spatial proximity into a graph.



ILLUSTRATIVE 3D POINT CLOUD

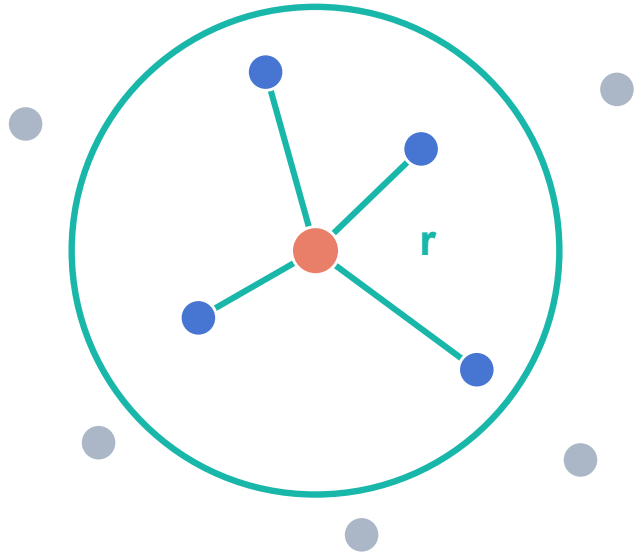
One parameter r defines a threshold graph

Make two distinct records adjacent when their Euclidean distance is at most r.

DISTANCE RULE

$$\text{distance}(p_i, p_j) \le r$$

- One vertex per input record
- Edges are undirected
- No self-loops; no repeated edges

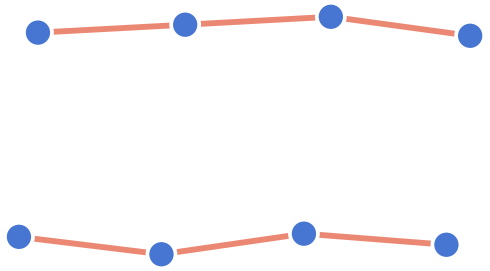


2D schematic of the 3D distance rule

The choice of radius changes the graph

Same eight vertices. Three different values of r. The edge sets are computed from the shown point locations.

Too small

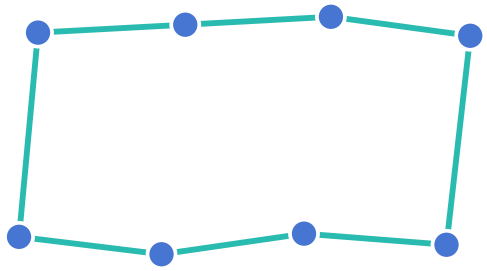


r = 1.06

6 edges

connected = 0

Connected + sparse

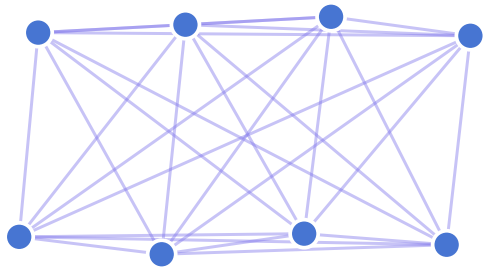


r = 1.36

8 edges

connected = 1

Very large



r = 3.60

28 edges

connected = 1

As r increases, edges never disappear; connectivity cannot be lost.

A4 is a graph evaluator—not a radius optimizer

Given a dataset and one chosen r, return two facts about the resulting graph.

INPUT

points[]

n points in 3D

radius r ≥ 0

→

graph_stats

build • query • analyze

→

OUTPUT

edges = 8

connected = 1

Illustrative input: the middle graph from the previous slide.

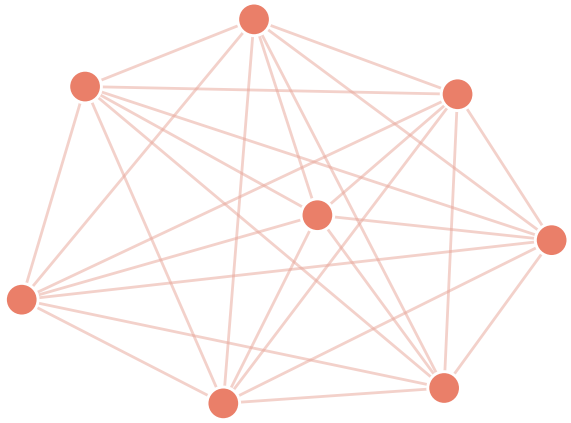
A caller can try several radii and look for a connected graph with an acceptable edge count.
You are not required to search for the best r.

Why not test the distance between every pair?

A brute-force approach compares all unordered pairs, including pairs that are obviously far apart.

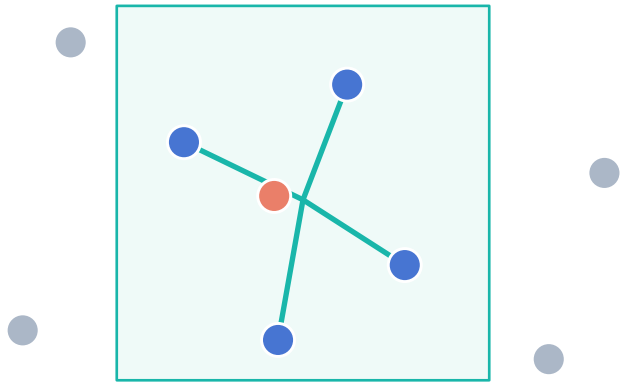
ALL-PAIRS DISTANCE CHECKS

n(n − 1) / 2 possible pairs



REGION-BASED CANDIDATE SEARCH

Check local candidates first



Example scale: n = 10,000 → 49,995,000 unordered pairs.

A k-d tree helps avoid many unnecessary checks for localized neighborhoods; dense graphs can still require quadratic work.

First query a cube. Then test true distance.

A cube is easy to search in a k-d tree; its candidates contain all points inside the radius-r sphere.

1 / RANGE QUERY

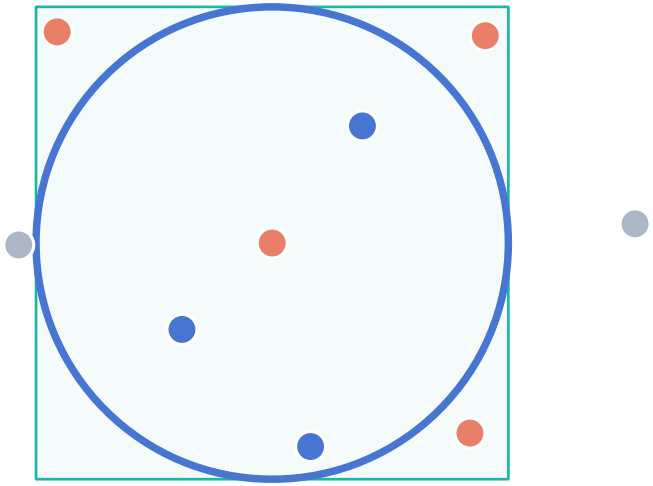
Closed cube, side length 2r

$$|x - \text{center.x}| \le r$$
$$|y - \text{center.y}| \le r$$
$$|z - \text{center.z}| \le r$$

INCLUDE THE BOUNDARY

2 / DISTANCE FILTER

Keep only true neighbors

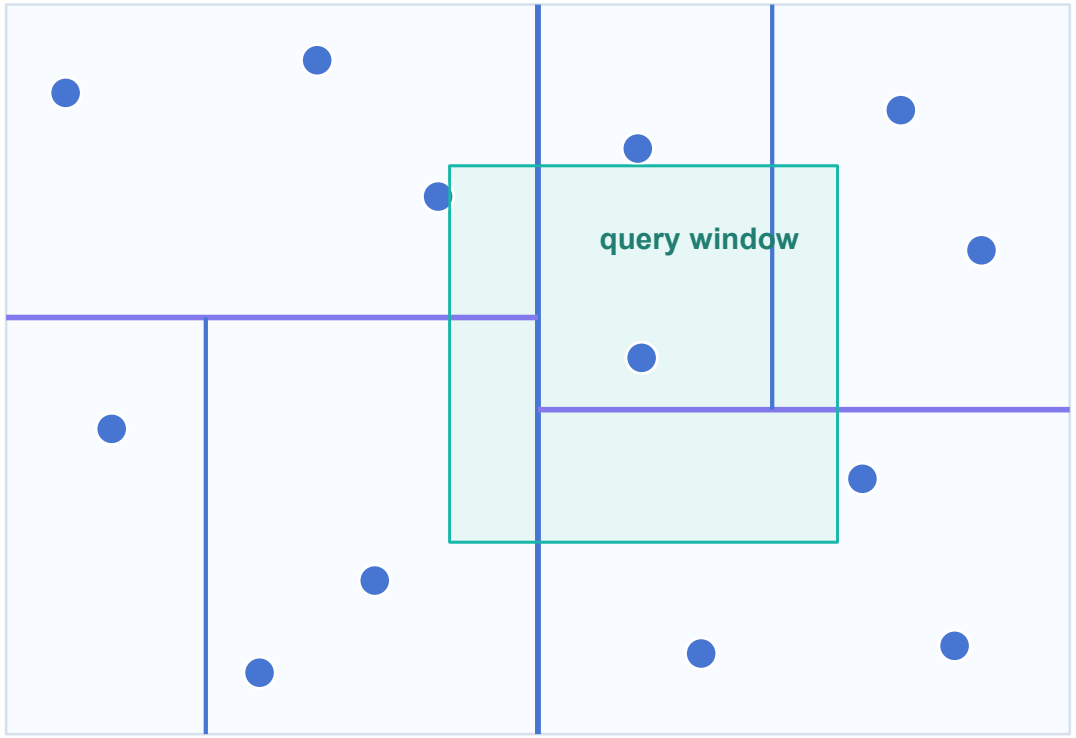


2D slice of a 3D cube and sphere

$$\Delta x^2 + \Delta y^2 + \Delta z^2 \le r^2 \quad (\Delta = \text{point} - \text{center})$$

A k-d tree can ignore whole spatial regions

The key advantage: spatial partitions let a range query skip subtrees that cannot overlap the cube.



2D partition schematic (the A4 tree is 3D)

Organize

Coordinate-based splits structure the search.

Reject

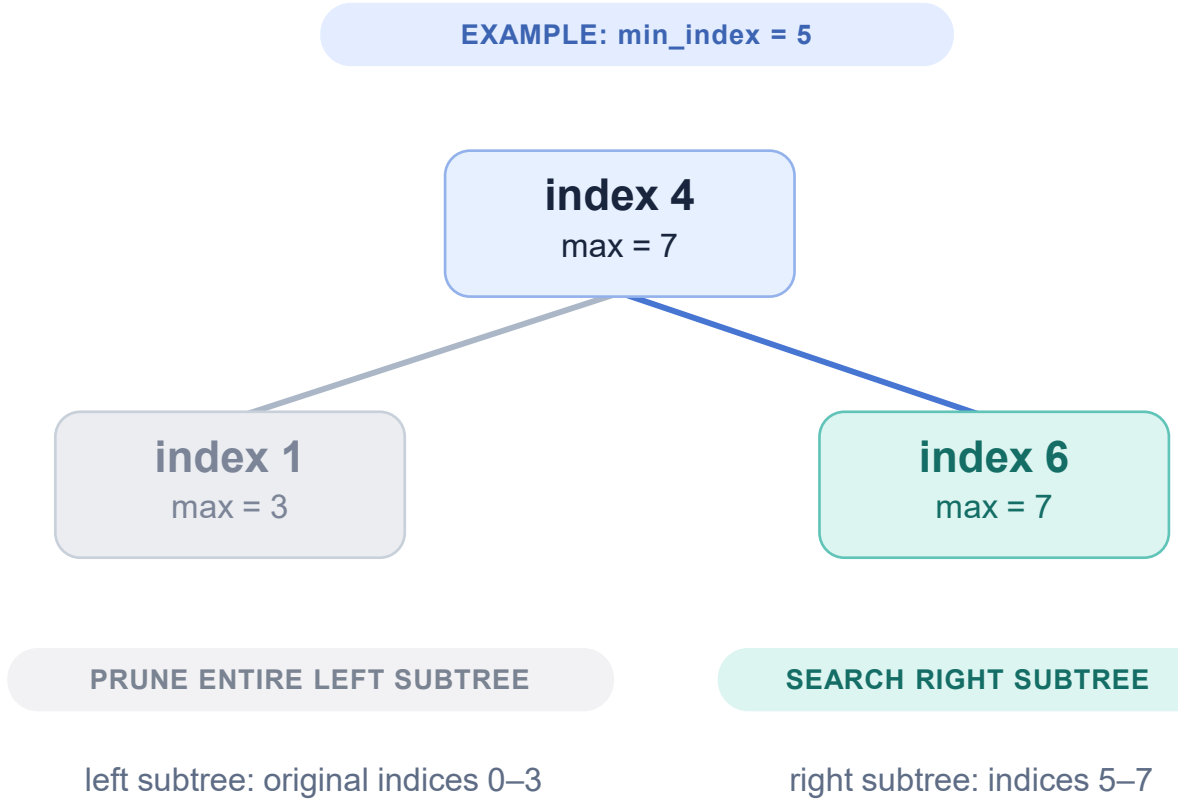
Skip subtrees whose spatial regions cannot intersect the query cube.

Inspect

Check only potentially relevant points in overlapping regions.

A4 adds a second pruning rule: original index

Every subtree stores the maximum original input index among its points.



INDEX PRUNING

If subtree_max < min_index, no point in that subtree is eligible.

Prune during tree search—not after gathering all spatial matches.

BOTH PRUNING RULES ARE REQUIRED

What kd_cube_query must return

Original input indices of all points meeting both the cube test and the index test.

```
kd_cube_query(tree, center, r, min_index, indices)
```

INDEX FILTER

$j \geq \text{min_index}$

X-COORDINATE

$|x_j - \text{center.x}| \leq r$

Y-COORDINATE

$|y_j - \text{center.y}| \leq r$

Z-COORDINATE

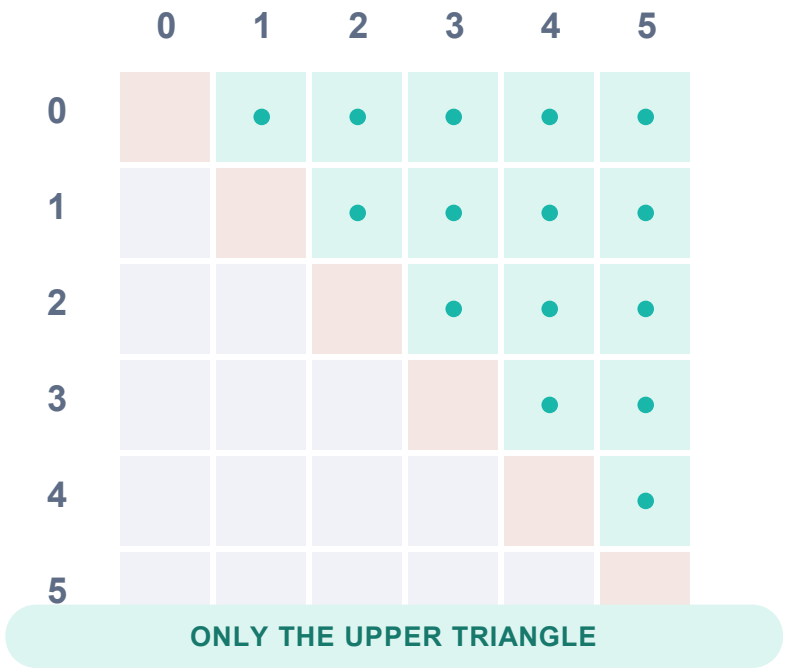
$|z_j - \text{center.z}| \leq r$

- Every eligible index, exactly once
- Include points on cube faces
- indices has capacity for n entries
- Any output order is acceptable
- Center need not be an input point
- If min_index ≥ n, return zero

Count every undirected edge once

When processing vertex i, query only original indices j > i.

PAIR ENUMERATION



FOR EACH INPUT INDEX I

```
min_index = i + 1
```

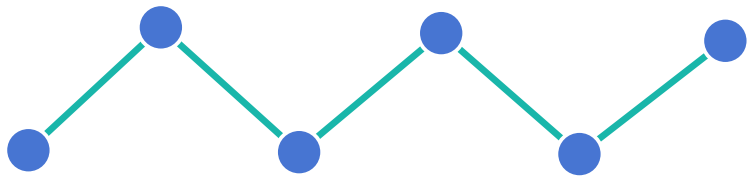
This skips self-pairs (i, i) and the reverse copy of each pair (j, i).

A cube hit is only a candidate: count an edge after the true distance test passes.

Edge count and connectivity are different facts

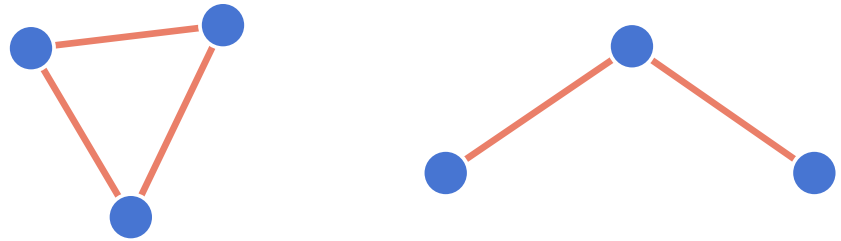
Two graphs can have the same number of edges, yet only one has a path between every pair of vertices.

CONNECTED



6 vertices · 5 edges · connected = 1

DISCONNECTED

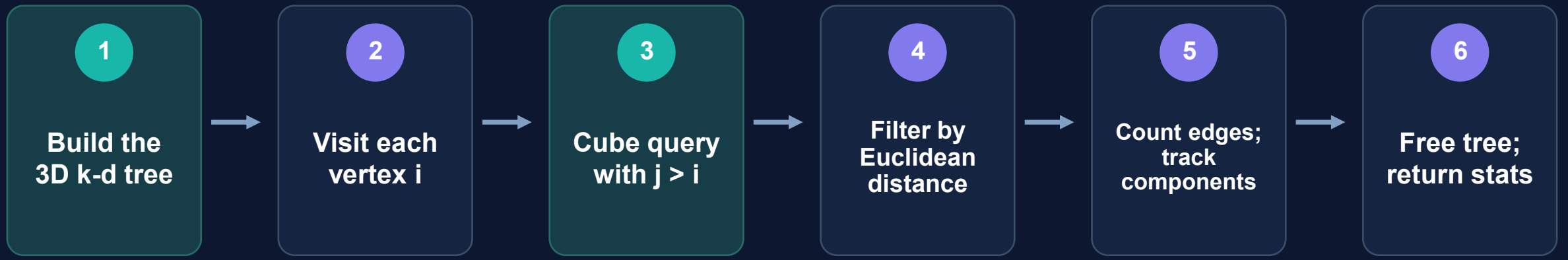


6 vertices · 5 edges · connected = 0

You may test connectivity with a graph traversal, union–find, or another suitable method.

What happens inside graph_stats?

Conceptually: construct a tree, enumerate local candidates, retain real edges, and determine connectivity.



K-D TREE IMPLEMENTATION: INDEPENDENT WORK

GRAPH-ANALYSIS LOGIC: AI ASSISTANCE PERMITTED

The graph routine calls your own k-d tree functions; it does not bypass them.

Four functions. One submitted source file.

The provided a4_api.h defines the interface; your a4.c must implement it unchanged.

REQUIRED FUNCTION SIGNATURES

```
KDTree *kd_build(const Point *points, size_t n);
size_t kd_cube_query(const KDTree *tree, Point center, double r,
                     size_t min_index, size_t *indices);
void kd_free(KDTree *tree);
GraphStats graph_stats(const Point *points, size_t n, double r);
```

Point: x, y, z (double)

GraphStats: edges, connected

No main() or input/output

Small details can change the answer

These conventions are part of the required behavior—not optional optimizations.

ONE VERTEX

n = 1 → 0 edges and connected = 1

DUPLICATE COORDINATES

Different records are different vertices, even at distance 0.

INCLUSIVE BOUNDARIES

Equal to r counts: cube faces and sphere surface.

ARBITRARY QUERY CENTER

A cube center need not be an input point.

INDEX PAST THE END

min_index ≥ n → zero query matches.

REPEATED CALLS

Different datasets and radii must work; release owned memory.

Inputs: n ≥ 1, finite coordinates, and finite r ≥ 0; distance arithmetic will not overflow.

There is a deliberate division of work

The k-d tree is the core independent data-structures exercise; graph analysis permits AI assistance.

INDEPENDENT — NO AI ASSISTANCE

- kd_build and its helpers
- kd_cube_query and both pruning mechanisms
- kd_free and memory management
- Any k-d tree structs, debugging, inspection, or refactoring

AI PERMITTED — GRAPH ANALYSIS ONLY

- graph_stats graph construction and edge counting
- Connectivity logic (such as union–find)
- Integrating with your own tree via the public API

AI cannot replace or modify the independent component.

The assignment specification must not be sent to AI

Limited permission for graph code does not permit sharing the assignment PDF or its contents.

Do not upload, paste, transcribe, screenshot, quote, or otherwise communicate the PDF—or any part of its contents—to an AI system.

SOLE EXCEPTION

You may share the separately supplied a4_api.h for permitted graph integration.

YOUR PROMPT

If using AI for graph_stats, independently compose your own prompt.

No AI assistance of any kind is permitted for k-d-tree code.

Your deliverable: one file, one graph utility

Use the assignment PDF and provided header for the authoritative details.

SUBMIT THROUGH CANVAS

a4.c

```
#include "a4_api.h"
```

All four public functions + any helpers

SUCCESS MEANS

- A genuine 3D k-d tree, implemented independently
- **Both spatial and subtree-index pruning**
- Correct cube matches, graph edges, and connectivity
- Correct memory use and repeated calls

Main idea: adapt a spatial search structure to make a graph-analysis problem practical.

SUBMISSION REQUIREMENTS

1. You will submit one – *and only one* – file: A4.c. There is no required report.

2. The TAs will be using a testing program to evaluate submitted code, so your A4.c must contain the specified functions with the specified prototypes.

3. You will need to perform your own testing to ensure your functions work correctly. This means you will need to create your own test datasets and verify your results. A good way to do this is to create a simpler brute-force implementation that you're confident is correct, and then compare its results to your results.

CRITICALLY IMPORTANT THINGS TO REMEMBER

1. You must implement the k-d tree components entirely on your own. You are not to use AI, and you are not to collaborate with anyone else or use code obtained elsewhere that you did not originate.

2. You may use an AI assistant for the remaining components, but you must design your own prompts. **You are not permitted to upload the assignment or these slides to any AI**. And, of course, you are not to collaborate with anyone else or use prompts that were not created by you.

3. Failure to adhere to one or more of these restrictions would constitute an act of academic dishonesty and result in a score of zero for the assignment.